Prerequisite

1. Understand image processing in Julia, please check YouTube Video: Working With Images in Julia | Week 1, lecture 3 | 18.S191 MIT Fall 2020 | Grant Sanderson.
2. Undertand fast Fourier transformation for image processing, please check the following videos by Steve Brunton:
   - The Fast Fourier Transform (FFT). It may require some knowledge about complex numbers.
   - Image Compression and the FFT
3. Understand tensor network, please check the following repository: Tutorial on Tensor Networks.
4. Understand basic optimization theory, please check:
   - The 3blue1brown video: Gradient descent, how neural networks learn | Deep Learning Chapter 2
   - Manifold optimization: YouTube video Manopt.jl: Optimisation on Riemannian Manifolds | Ronny Bergmann | JuliaCon 2022 and Julia package Manopt.jl.

Get started

1. Go through the code in `examples/img_process.jl`. It may require some knowledge about manifold optimization, please check the documentation page of Manifolds.jl.
2. Read the note `note/main.typ` to understand the theory underlying the code.

Code Structure

The codebase evolves through three stages, each solving problems from the previous one. The file tree below shows which files belong to which stage:

```
src/
├── ParametricDFT.jl          # Main module: exports, include() ordering
├── qft.jl                    # QFT circuit → einsum contraction code
├── loss.jl                   # Loss functions: L1Norm, L2Norm, MSELoss
├── basis.jl                  # AbstractSparseBasis: QFTBasis,
│                             #   EntangledQFTBasis, TEBDBasis
├── entangled_qft.jl          # Entangled QFT circuit (XY correlation)
├── tebd.jl                   # TEBD circuit (brickwork topology)
├── manifolds.jl              # Riemannian manifolds: UnitaryManifold,
│                             #   PhaseManifold, batched linear algebra
├── optimizers.jl             # RiemannianGD (Armijo), RiemannianAdam
├── training.jl               # Training pipeline, checkpointing,
│                             #   early stopping
├── compression.jl            # Image compression using learned bases
├── serialization.jl          # Basis save/load (JSON3)
└── visualization.jl          # Training loss plots (CairoMakie)

ext/
└── CUDAExt.jl                # GPU: batched CUBLAS, 2×2 inverse formula

test/
├── runtests.jl               # Test runner
├── basis_tests.jl            # Basis construction + transform tests
├── loss_tests.jl             # Loss function correctness
├── manifold_tests.jl         # Manifold projection/retraction/transport
├── optimizer_tests.jl        # Optimizer convergence tests
├── training_tests.jl         # Training pipeline integration
├── compression_tests.jl      # Compression round-trip tests
├── entangled_qft_tests.jl    # Entangled QFT tests
├── tebd_tests.jl             # TEBD circuit tests
```

```
├── serialization_tests.jl    # Save/load round-trip tests
└── cuda_tests.jl             # GPU-specific tests (requires CUDA)
```

Stage 1: Manopt.jl Baseline

The Approach

Use Manopt.jl to optimize QFT circuit parameters on a product manifold. All circuit tensors are 2×2 unitary matrices, so we model each as a point on the Stiefel manifold `Stiefel(2, 2, ℂ)` (which equals $U(2)$). The tensors are packed into a `ProductManifold` and optimized via `gradient_descent`.

Note: Manopt.jl is *not* used in the main library (`src/`). It is only used in `examples/manopt_baseline.jl` as a comparison benchmark. The main training pipeline (`src/training.jl`) always uses the custom `RiemannianGD` / `RiemannianAdam` optimizers.

```julia
# Build product manifold: all tensors on Stiefel(2,2,ℂ) ≅ U(2)
# Note: Stiefel is used instead of UnitaryMatrices to avoid
# a local_metric MethodError in Manopt's Armijo linesearch.
S = Stiefel(2, 2, ℂ)
M = ProductManifold(ntuple(_ -> S, length(tensors))...)

# Convert between tensors and manifold points
p0 = ArrayPartition(tensors...)          # tensors → point
ts = collect(p.x)                        # point → tensors

# Loss: average over dataset, Riemannian gradient via Zygote AD
f(M, p) = mean(loss_function(collect(p.x), m, n, optcode,
                             img, loss) for img in images)
grad_f(M, p) = ManifoldDiff.gradient(M, x -> f(M, x), p,
    RiemannianProjectionBackend(AutoZygote()))

# Run optimization
result = gradient_descent(M, f, grad_f, p0;
    stopping_criterion = StopAfterIteration(steps) |
                         StopWhenGradientNormLess(1e-5))
```

What Manopt.jl Provides

Manopt.jl is a mature library with 35+ solvers. For our use case, the relevant features are:

- `gradient_descent` with **Armijo line search by default**, plus Wolfe-Powell, cubic bracketing, nonmonotone line search options
- Direction update rules: momentum, Nesterov acceleration, gradient averaging
- Stochastic gradient descent (single-sample updates)
- Automatic differentiation via ManifoldDiff.jl + Zygote

Problems Encountered

Problem	Why It Happens
Slow: per-tensor manifold operations	Manopt.jl's `ProductManifold` iterates over each component manifold one-by-one. For a circuit with K tensors, this means K separate projection/retraction calls — no way to batch same-type tensors into a single operation.

Cannot use GPU	`ProductManifold` uses `ArrayPartition` with heterogeneous element types. CUDA requires homogeneous `CuArray`. Additionally, individual 2×2 matrix operations are too small for GPU kernels (launch overhead 10μs dominates).
No Riemannian Adam	Manopt.jl does not include Adam or any adaptive learning rate optimizer. Its gradient-based methods are limited to GD with momentum/Nesterov variants. For our problem, Adam's per-parameter adaptive rates converge faster than GD.

These problems motivate Stage 2: building custom Riemannian optimizers with batched operations.

Stage 2: Custom Riemannian Optimizers

Notation

We use the following notation consistently in this section:

- $\boldsymbol{\theta}_k \in \mathbb{C}^{d \times d}$: the k-th gate tensor (circuit parameter), $k = 1, ..., K$
- $\boldsymbol{X} \in \mathbb{C}^{2 \times \cdots \times 2}$: input image reshaped to $m + n$ qubit dimensions (each of size 2)
- $\boldsymbol{Y}$: output of the forward transform
- $\mathcal{L}$: loss function value
- $\boldsymbol{g}_k$: Euclidean gradient of $\mathcal{L}$ with respect to $\boldsymbol{\theta}_k$
- $\tilde{\boldsymbol{g}}_k = \mathrm{proj}(\boldsymbol{\theta}_k, \boldsymbol{g}_k)$: Riemannian gradient (projection of $\boldsymbol{g}_k$ onto the tangent space at $\boldsymbol{\theta}_k$)
- $\mathrm{Retr}_{\boldsymbol{\theta}}(\boldsymbol{\xi})$: retraction from $\boldsymbol{\theta}$ along tangent vector $\boldsymbol{\xi}$ (maps back to the manifold)
- $\Gamma_{\boldsymbol{\theta} \to \boldsymbol{\theta}'}$: parallel transport from tangent space at $\boldsymbol{\theta}$ to tangent space at $\boldsymbol{\theta}'$
- α: step size (learning rate)

Problem → Solution Overview

Problem from Stage 1	Solution	File
Per-tensor manifold ops	Group same-type tensors into (d, d, K) batches, apply manifold ops once per type	`manifolds.jl`
No Riemannian Adam	Custom `RiemannianAdam` with momentum transport + `RiemannianGD` with Armijo line search	`optimizers.jl`
No GPU support	Batched linear algebra dispatching to CUBLAS on GPU; device abstraction via `to_device`	`ext/CUDAExt.jl`
Single-image einsum contraction	Batched einsum: append batch label to the contraction so B images share one kernel call	`loss.jl`

Solution 1: Batched Manifold Operations (`manifolds.jl`)

Key insight: Circuit tensors are either $U(2)$ unitaries or $U(1)^4$ phase gates. Group all same-type tensors into a single 3D array and apply manifold operations to the whole batch.

```
Before (Manopt.jl):                 After (custom):
  for each tensor:                    group_by_manifold(tensors)
    project(manifold_i, tensor_i)     stack_tensors → (d, d, K)
```

```
    retract(manifold_i, tensor_i)    project(manifold, batch)   # one call
                                     retract(manifold, batch)   # one call
                                     unstack_tensors!
```

Two manifold types implemented:

`UnitaryManifold` — $U(n)$ unitary group:
- Projection: $\mathrm{proj}(\boldsymbol{U}, \boldsymbol{G}) = \boldsymbol{U} \cdot \mathrm{skew}\big(\boldsymbol{U}^\dagger \boldsymbol{G}\big)$, where $\mathrm{skew}(\boldsymbol{A}) = \big(\boldsymbol{A} - \boldsymbol{A}^\dagger\big)/2$
- Retraction (Cayley): $\mathrm{Retr}_{\boldsymbol{U}}(\boldsymbol{\xi}) = \big(\boldsymbol{I} - \frac{\alpha}{2} \cdot \boldsymbol{W}\big)^{-1}\big(\boldsymbol{I} + \frac{\alpha}{2} \cdot \boldsymbol{W}\big) \cdot \boldsymbol{U}, \quad \boldsymbol{W} = \boldsymbol{\xi}\boldsymbol{U}^\dagger$
- Transport: $\Gamma_{\boldsymbol{U} \to \boldsymbol{U}'}(\boldsymbol{v}) = \mathrm{proj}(\boldsymbol{U}', \boldsymbol{v})$ (re-project onto new tangent space)

`PhaseManifold` — $U(1)^d$ product of unit circles:
- Projection: $\mathrm{proj}(\boldsymbol{z}, \boldsymbol{g}) = \mathrm{i} \cdot \mathrm{Im}(\overline{\boldsymbol{z}} \odot \boldsymbol{g}) \odot \boldsymbol{z} \quad (\mathrm{i} = \sqrt{-1})$
- Retraction: $\mathrm{Retr}_{\boldsymbol{z}}(\boldsymbol{\xi}) = (\boldsymbol{z} + \alpha\boldsymbol{\xi}) \oslash |\boldsymbol{z} + \alpha\boldsymbol{\xi}|$ (element-wise normalize)
- Transport: $\Gamma_{\boldsymbol{z} \to \boldsymbol{z}'}(\boldsymbol{v}) = \mathrm{proj}(\boldsymbol{z}', \boldsymbol{v})$

```julia
# Classify and batch tensors ONCE before training loop
manifold_groups = group_by_manifold(tensors)  # Dict{Manifold, Vector{Int}}
for (manifold, indices) in manifold_groups
    pb = stack_tensors(tensors, indices)       # (d, d, K) batch
    rg = project(manifold, pb, grad_batch)     # batched projection
    new_pb = retract(manifold, pb, .-rg, α)    # batched retraction
    unstack_tensors!(tensors, new_pb, indices)  # write back
end
```

Solution 2: Custom Optimizers (`optimizers.jl`)

`RiemannianGD` with Armijo Line Search

Gradient descent with adaptive step size. At each iteration, compute the Riemannian gradient $\tilde{\boldsymbol{g}}$ and search for a step size α satisfying the Armijo sufficient decrease condition:

$$\mathcal{L}(\mathrm{Retr}_{\boldsymbol{\theta}}(-\alpha\tilde{\boldsymbol{g}})) \leq \mathcal{L}(\boldsymbol{\theta}) - c \cdot \alpha \cdot \|\tilde{\boldsymbol{g}}\|^2$$

The algorithm tries $\alpha, \alpha\tau, \alpha\tau^2, \ldots$ (backtracking with factor $\tau \in (0, 1)$) until the condition holds. All retraction trials use batched operations.

`RiemannianAdam` with Momentum Transport

Extends Adam to manifolds (Bécigneul & Ganea, 2019). The key challenge: after retraction, the first moment $\boldsymbol{m}_t$ lives in the *old* tangent space and must be transported to the *new* one.

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\tilde{\boldsymbol{g}}_t, \qquad \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2)\ |\tilde{\boldsymbol{g}}_t|^2$$

Bias-corrected direction: $\boldsymbol{d}_t = \frac{\boldsymbol{m}_t/(1-\beta_1^t)}{\sqrt{\boldsymbol{v}_t/(1-\beta_2^t)+\varepsilon}}$

Update and transport: $\boldsymbol{\theta}_{t+1} = \mathrm{Retr}_{\boldsymbol{\theta}_t}(-\alpha\boldsymbol{d}_t), \qquad \boldsymbol{m}_t \leftarrow \Gamma_{\boldsymbol{\theta}_t \to \boldsymbol{\theta}_{t+1}}(\boldsymbol{m}_t)$

```julia
# Per-manifold Adam update (fused broadcasts over (d,d,K) arrays)
@. m_state = β₁ * m_state + (1 - β₁) * rg         # moment update
@. v_state = β₂ * v_state + (1 - β₂) * abs2(rg)    # variance update
@. dir = (m_state / bc1) / (√(v_state / bc2) + ε)  # bias-corrected
new_batch = retract(manifold, old_batch, .-dir, lr) # retract
m_state = transport(manifold, old_batch, new_batch, m_state)  # transport
```

Solution 3: Batched Multi-Image Einsum (`loss.jl`)

Background: OMEinsum and the Forward Transform

The QFT circuit is represented as a tensor network contraction using OMEinsum.jl. In OMEinsum, a contraction is specified by a `DynamicEinCode`: a list of input index lists and an output index list. Indices that appear in inputs but not in the output are summed over (contracted). For example, matrix multiplication $\boldsymbol{C}_{i,k} = \sum_j \boldsymbol{A}_{i,j}\boldsymbol{B}_{j,k}$ is written as:

```
ein"ij, jk -> ik"(A, B)    or equivalently    DynamicEinCode([[1,2], [2,3]], [1,3])
```

where integer labels replace characters (OMEinsum uses `Int` labels internally).

The function `qft_code(m, n)` builds the einsum for the QFT circuit:

1. Construct the QFT circuit using `Yao.EasyBuild.qft_circuit` for m row qubits and n column qubits
2. Convert the circuit to an einsum via `yao2einsum` — this assigns each gate and each qubit wire an integer index label
3. Optimize the contraction order using `TreeSA()` (done once, reused for all iterations)

Concrete example ($m = 2, n = 2$, i.e. 4×4 images with 4 qubits). The circuit has $K = 6$ gate tensors (4 Hadamard gates H and 2 controlled-phase gates M_k), plus the image tensor. After `yao2einsum`, the contraction looks like:

```
DynamicEinCode([[5,1], [6,2], [7,3], [8,4], [5,2], [7,4], [5,6,7,8]], [1,2,3,4])
```

Reading this notation:

- Gate tensors: $\boldsymbol{\theta}_1$ has indices $[5, 1]$, $\boldsymbol{\theta}_2$ has $[6, 2]$, ..., $\boldsymbol{\theta}_6$ has $[7, 4]$ — each is a 2×2 matrix
- Image tensor: indices $[5, 6, 7, 8]$ — these are the $m + n = 4$ qubit dimensions
- Output: indices $[1, 2, 3, 4]$ — the transformed image's qubit dimensions
- Contracted indices: $5, 6, 7, 8$ appear in both gate and image inputs but not in the output, so they are summed over

The corresponding tensor contraction is:

$$\boldsymbol{Y}_{j_1,j_2,j_3,j_4} = \sum_{i_1,i_2,i_3,i_4} (\boldsymbol{\theta}_1)_{i_1,j_1} (\boldsymbol{\theta}_2)_{i_2,j_2} (\boldsymbol{\theta}_3)_{i_3,j_3} (\boldsymbol{\theta}_4)_{i_4,j_4} (\boldsymbol{\theta}_5)_{i_1,i_2} (\boldsymbol{\theta}_6)_{i_3,i_4} \boldsymbol{X}_{i_1,i_2,i_3,i_4}$$

where we used the label mapping: $5 \to i_1, 6 \to i_2, 7 \to i_3, 8 \to i_4$ (contracted) and $1 \to j_1, 2 \to j_2, 3 \to j_3, 4 \to j_4$ (output). In general, for m row qubits and n column qubits:

$$\boldsymbol{Y}_{j_1,\ldots,j_{m+n}} = \sum_{i_1,\ldots,i_{m+n}} \prod_{k=1}^{K} (\boldsymbol{\theta}_k)_{a_k,b_k} \cdot \boldsymbol{X}_{i_1,\ldots,i_{m+n}}$$

where each gate $\boldsymbol{\theta}_k$ connects a pair of indices (a_k, b_k) determined by the circuit structure, and all i_ℓ that do not appear in the output are summed over.

The Problem: Single-Image Contraction

Without batching, training on a dataset of N images requires calling this einsum N times per gradient step — each call launches its own kernel(s).

The diagram below shows the tensor network for *one* image. The gate tensors $\boldsymbol{\theta}_k$ (shared across all images) each have 2 legs that connect into the circuit, while the image tensor $\boldsymbol{X}$ feeds $m + n$ qubit indices through the circuit to produce output $\boldsymbol{Y}$:

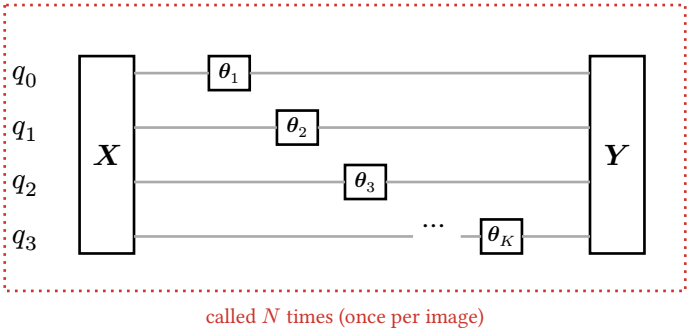

Figure 1: Single-image einsum: gate tensors $\boldsymbol{\theta}_k$ sit on qubit lines between input $\boldsymbol{X}$ and output $\boldsymbol{Y}$. This contraction is called N times, once per image.

This is inefficient because the contraction tree is *identical* for every image — only the image tensor (the last entry `[5,6,7,8]`) changes. The gate tensors $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K$ are shared across all images, yet `optcode(tensors..., reshape(img, ...))` must be called N times.

The Solution: Append a Batch Dimension

The key insight: add a new *batch label* to the image input and output index lists, leaving gate indices unchanged. Since this label is not contracted (it appears in both the image input and the output), OMEinsum processes all B images in a single kernel call.

Continuing the $m = 2, n = 2$ example, `make_batched_code` computes `batch_label = max(1,...,8) + 1 = 9` and appends it:

```
DynamicEinCode([[5,1], [6,2], [7,3], [8,4], [5,2], [7,4], [5,6,7,8,9]], [1,2,3,4,9])
```

Only the last two entries change (bold): the image input gains label 9, the output gains label 9. All gate index lists are unchanged. The contraction becomes:

$$\boldsymbol{Y}_{j_1,j_2,j_3,j_4,b} = \sum_{i_1,i_2,i_3,i_4} (\boldsymbol{\theta}_1)_{i_1,j_1} \cdots (\boldsymbol{\theta}_6)_{i_3,i_4} \boldsymbol{X}_{i_1,i_2,i_3,i_4,b}$$

where $b \in \{1, \ldots, B\}$ indexes images in the batch. Because b is a free index (not summed over), the gate contractions are identical for every image — OMEinsum handles this in one call. In general:

$$\boldsymbol{Y}_{j_1,\ldots,j_{m+n},b} = \sum_{i_1,\ldots,i_{m+n}} \prod_{k=1}^{K} (\boldsymbol{\theta}_k)_{a_k,b_k} \cdot \boldsymbol{X}_{i_1,\ldots,i_{m+n},b}$$

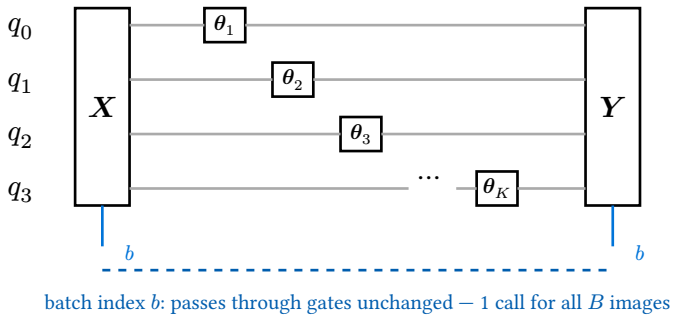

Figure 2: Batched einsum: an extra batch index b (blue) is added to $\boldsymbol{X}$ and $\boldsymbol{Y}$. Gate tensors $\boldsymbol{\theta}_k$ have no b index, so the *same* contraction processes all B images in one call.

The image tensor $\boldsymbol{X}$ has shape $(2, 2, \ldots, 2, B) \in \mathbb{C}^{2^{m+n} \times B}$ — B images stacked along a new last dimension.

The implementation in `loss.jl` has three steps:

```
# Step 1: Append batch label to the einsum code
# make_batched_code finds max label (=8), sets batch_label = 9,
# appends it to image input [5,6,7,8] → [5,6,7,8,9]
# and output [1,2,3,4] → [1,2,3,4,9]
batched_flat, batch_label = make_batched_code(optcode, n_gates)

# Step 2: Re-optimize contraction order for batched tensor sizes
# size_dict: {1=>2, 2=>2, ..., 8=>2, 9=>B}  (all qubit dims = 2, batch dim = B)
batched_optcode = optimize_batched_code(batched_flat, batch_label, batch_size)

# Step 3: Stack B images and evaluate in one einsum call
# Each image (2^m × 2^n) is reshaped to (2,2,...,2) then stacked along dim m+n+1
stacked = cat([reshape(img, fill(2,m+n)...) for img in batch]...; dims=m+n+1)
result = batched_optcode(tensors..., stacked)  # shape: (2,2,...,2,B)
```

Loss Dispatch by Type

The batching strategy differs by loss function:

- **L1/L2**: Fully batched end-to-end. One einsum call produces all B outputs as a $(2, \ldots, 2, B)$ tensor, then a single `sum(abs.())` or `sum(abs2.())` reduction:

$$\mathcal{L}_{\mathrm{L1}} = \frac{1}{B}\sum_{b=1}^{B}\sum_{j_1,\ldots,j_{m+n}} |\boldsymbol{Y}_{j_1,\ldots,j_{m+n},b}|, \qquad \mathcal{L}_{\mathrm{L2}} = \frac{1}{B}\sum_{b=1}^{B}\sum_{j_1,\ldots,j_{m+n}} |\boldsymbol{Y}_{j_1,\ldots,j_{m+n},b}|^2$$

- **MSE**: The forward pass is batched (one einsum call), but `topk_truncate` must run per-image because the truncation mask depends on each image's frequency content. The inverse transform also runs per-image. So MSE is "half-batched": batched forward, sequential truncation + inverse:

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{B}\sum_{b=1}^{B} \| \boldsymbol{X}_b - \boldsymbol{Y}_b^{-1} \|^2, \qquad \text{where } \boldsymbol{Y}_b^{-1} = \mathrm{einsum}^{-1}(\boldsymbol{\theta}_1^*, \ldots, \boldsymbol{\theta}_K^*, \mathrm{truncate}_k(\boldsymbol{Y}_b))$$

Here einsum^{-1} denotes the inverse transform (using conjugated tensors $\boldsymbol{\theta}_k^*$), and $\mathrm{truncate}_k$ keeps the top-k frequency-weighted coefficients of $\boldsymbol{Y}_b$.

How Everything Fits Together: The Training Loop

The diagram below shows one iteration of the training loop. The gate tensors and image batch feed into the batched einsum; the output flows through the loss, then Zygote differentiates back to get gradients, which the Riemannian optimizer uses to update the tensors on the manifold:

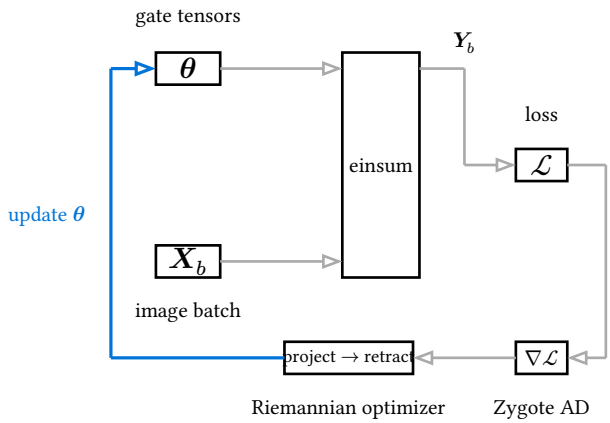

Figure 3: One training iteration. Forward: gate tensors $\boldsymbol{\theta}$ and images $\boldsymbol{X}_b$ contract via batched einsum to produce $\boldsymbol{Y}_b$, then the loss $\mathcal{L}$ is evaluated. Backward: Zygote computes $\nabla\mathcal{L}$, the Riemannian optimizer projects the gradient and retracts to update $\boldsymbol{\theta}$ on the manifold.

Remaining Problem

The batched operations run on CPU with per-slice loops:

```
batched_matmul: for k in 1:K; mul!(C[:,:,k], A[:,:,k], B[:,:,k]); end
batched_inv:    for k in 1:K; C[:,:,k] = inv(A[:,:,k]); end
```

For 2×2 matrices, each operation takes nanoseconds but each kernel launch takes 10μs. On GPU, K separate launches would be slower than CPU. This motivates Stage 3.

Stage 3: GPU Acceleration

Problem → Solution Overview

Problem from Stage 2	Solution	File
K separate kernel launches for K matrix ops	`CUBLAS.gemm_strided_batched!` — single kernel for all K multiplications	`ext/CUDAExt.jl`
Per-slice `inv()` too small for GPU	Closed-form 2×2 inverse: $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = (ad - bc)^{-1} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$	`ext/CUDAExt.jl`
CPU-only einsum	`CuArray` dispatch — OMEinsum automatically uses GPU contraction	`ext/CUDAExt.jl`
Device management boilerplate	`to_device(x, :gpu/:cpu)` abstraction	`ext/CUDAExt.jl`

GPU Dispatch Table

On GPU, the batched operations dispatch to optimized kernels:

Operation	CPU (manifolds.jl)	GPU (CUDAExt.jl)
`batched_matmul`	Per-slice `mul!` loop	`CUBLAS.gemm_strided_batched!` (1 kernel)
`batched_inv`	Per-slice `inv()` loop	Broadcast 2×2 formula (1 kernel)
`batched_adjoint`	`permutedims(conj.())`	Same (GPU broadcast)
Einsum	CPU OMEinsum	CuArray dispatch (GPU contraction)

When GPU Wins vs CPU

- **Small circuits** (< 10 tensors): CPU faster due to GPU transfer overhead
- **Large circuits or batches**: GPU wins from batched parallelism
- **L1/L2 loss**: fully batched, GPU-friendly
- **MSE loss**: `topk_truncate` is per-image, limiting GPU parallelism

Sparse Basis Types

Three basis types are implemented, each a subtype of `AbstractSparseBasis`:

Basis Type	Circuit Structure	Parameters
`QFTBasis`	Standard QFT: Hadamard H + controlled-phase M_k gates, independent row/column qubits	$H \in U(2)$, $M_k \in U(1)^4$
`EntangledQFTBasis`	QFT + entanglement gates E_k between row/column qubits (captures XY correlation)	QFT params + n additional phases φ_k

`TEBDBasis`	Ring topology of Hadamard + controlled-phase gates (brickwork pattern)	$H \in U(2), T_k \in U(1)^4$

Frequency-dependent truncation (`topk_truncate`) for compression. The score for coefficient (i, j) is:

$$s_{i,j} = |\boldsymbol{Y}_{i,j}| \cdot (1 + w_{i,j}), \qquad w_{i,j} = 1 - \frac{d_{i,j}}{2d_{\max}}$$

where $d_{i,j} = \sqrt{(i - i_c)^2 + (j - j_c)^2}$ is the distance from the DC component (center), and $d_{\max}$ is the maximum such distance. The top-k coefficients by score $s_{i,j}$ are kept; the rest are zeroed. This weighting favors low-frequency components (structural information) over high-frequency ones (fine details) of similar magnitude.

Running Examples

Optimizer Benchmark

Run `examples/optimizer_benchmark.jl` to compare:

- Manopt.jl GD (CPU baseline)
- PDFT `RiemannianGD` (CPU/GPU)
- PDFT `RiemannianAdam` (CPU/GPU)

Image Compression

Run `examples/basis_demo.jl` for a complete image compression workflow:

1. Load image and create a sparse basis (`QFTBasis`, `EntangledQFTBasis`, or `TEBDBasis`)
2. Train the basis on the image using `train_basis!`
3. Compress the image using `compress_image`
4. Compare reconstruction quality with the standard Fourier basis

Tasks

- Create an image dataset $\mathcal{D} = \{\boldsymbol{X}_b\}_{b=1}^N$
- Train the tensor network on the dataset and compare with Fourier basis
- Use GPU to speed up training (see `ext/CUDAExt.jl`)
- Experiment with different loss functions (L1, MSE, Hybrid)
- Experiment with different basis types (QFT, Entangled QFT, TEBD)
- Add edge detection features
- Compare compression quality at different compression ratios (60%, 70%, 95%)